

Dataset link:

<https://www.kaggle.com/datasets/adityajn105/flickr8k>

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Basic libraries

```
import os
import pickle
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import warnings
warnings.filterwarnings('ignore')
from math import ceil
from collections import defaultdict
from tqdm.notebook import tqdm          # Progress bar library for
Jupyter Notebook
```

Deep learning framework for building and training models

```
import tensorflow as tf
## Pre-trained model for image feature extraction
from tensorflow.keras.applications.vgg16 import VGG16,
preprocess_input
from tensorflow.keras.preprocessing.image import load_img,
img_to_array
```

Tokenizer class for captions tokenization

```
from tensorflow.keras.preprocessing.text import Tokenizer
```

Function for padding sequences to a specific length

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

Class for defining Keras models

```
from tensorflow.keras.models import Model
from tensorflow.keras.utils import to_categorical, plot_model
from tensorflow.keras.layers import Input, Dense, LSTM, Embedding,
Dropout, concatenate, Bidirectional, Dot, Activation, RepeatVector,
Multiply, Lambda
```

For checking score

```
from nltk.translate.bleu_score import corpus_bleu
```

Setting the input and output directory

```
INPUT_DIR = '/content/drive/MyDrive/Image Caption Generator using Deep
```

```
Learning on Flickr8K dataset/Images'
OUTPUT_DIR = '/content/drive/MyDrive/Image Caption Generator using
Deep Learning on Flickr8K dataset/Outputs'
```

```
# We are going to use pretrained vgg model
```

```
# Load the vgg16 model
```

```
model = VGG16()
```

```
# Restructuring the model to remove the last classification layer,
this will give us access to the output features of the model
```

```
model = Model(inputs=model.inputs, outputs=model.layers[-2].output)
```

```
# Printing the model summary
```

```
print(model.summary())
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels.h5
553467096/553467096 _____ 6s 0us/step
```

```
Model: "functional"
```

Layer (type) Param #	Output Shape
input_layer (InputLayer)	(None, 224, 224, 3)
0	
block1_conv1 (Conv2D)	(None, 224, 224, 64)
1,792	
block1_conv2 (Conv2D)	(None, 224, 224, 64)
36,928	
block1_pool (MaxPooling2D)	(None, 112, 112, 64)
0	
block2_conv1 (Conv2D)	(None, 112, 112, 128)
73,856	
block2_conv2 (Conv2D)	(None, 112, 112, 128)
147,584	

0	block2_pool (MaxPooling2D)	(None, 56, 56, 128)
295,168	block3_conv1 (Conv2D)	(None, 56, 56, 256)
590,080	block3_conv2 (Conv2D)	(None, 56, 56, 256)
590,080	block3_conv3 (Conv2D)	(None, 56, 56, 256)
0	block3_pool (MaxPooling2D)	(None, 28, 28, 256)
1,180,160	block4_conv1 (Conv2D)	(None, 28, 28, 512)
2,359,808	block4_conv2 (Conv2D)	(None, 28, 28, 512)
2,359,808	block4_conv3 (Conv2D)	(None, 28, 28, 512)
0	block4_pool (MaxPooling2D)	(None, 14, 14, 512)
2,359,808	block5_conv1 (Conv2D)	(None, 14, 14, 512)
2,359,808	block5_conv2 (Conv2D)	(None, 14, 14, 512)
2,359,808	block5_conv3 (Conv2D)	(None, 14, 14, 512)

0	block5_pool (MaxPooling2D)	(None, 7, 7, 512)
0	flatten (Flatten)	(None, 25088)
102,764,544	fc1 (Dense)	(None, 4096)
16,781,312	fc2 (Dense)	(None, 4096)

Total params: 134,260,544 (512.16 MB)

Trainable params: 134,260,544 (512.16 MB)

Non-trainable params: 0 (0.00 B)

None

```
# Initialize an empty dictionary to store image features
image_features = {}

# Define the directory path where images are located
img_dir = os.path.join(INPUT_DIR, '/content/drive/MyDrive/Image
Caption Generator using Deep Learning on Flickr8K dataset/Images')

# Loop through each image in the directory
for img_name in tqdm(os.listdir(img_dir)):
    # Load the image from file
    img_path = os.path.join(img_dir, img_name)
    image = load_img(img_path, target_size=(224, 224))
    # Convert image pixels to a numpy array
    image = img_to_array(image)
    # Reshape the data for the model
    image = image.reshape((1, image.shape[0], image.shape[1],
image.shape[2]))
    # Preprocess the image for ResNet50
    image = preprocess_input(image)
    # Extract features using the pre-trained ResNet50 model
    image_feature = model.predict(image, verbose=0)
    # Get the image ID by removing the file extension
    image_id = img_name.split('.')[0]
    # Store the extracted feature in the dictionary with the image ID
as the key
    image_features[image_id] = image_feature
```

```

{"model_id":"ff1dd2be55e74ecb90bfa2661ed23962","version_major":2,"version_minor":0}

# Store the image features in pickle
pickle.dump(image_features, open(os.path.join(OUTPUT_DIR,
'img_features.pkl'), 'wb'))

# Load features from pickle file
pickle_file_path = os.path.join(OUTPUT_DIR, 'img_features.pkl')
with open(pickle_file_path, 'rb') as file:
    loaded_features = pickle.load(file)

with open(os.path.join(INPUT_DIR, '/content/drive/MyDrive/Image
Caption Generator using Deep Learning on Flickr8K
dataset/captions.txt'), 'r') as file:
    next(file)
    captions_doc = file.read()

# Create mapping of image to captions
image_to_captions_mapping = defaultdict(list)

# Process lines from captions_doc
for line in tqdm(captions_doc.split('\n')):
    # Split the line by comma(,)
    tokens = line.split(',')
    if len(tokens) < 2:
        continue
    image_id, *captions = tokens
    # Remove extension from image ID
    image_id = image_id.split('.')[0]
    # Convert captions list to string
    caption = " ".join(captions)
    # Store the caption using defaultdict
    image_to_captions_mapping[image_id].append(caption)

# Print the total number of captions
total_captions = sum(len(captions) for captions in
image_to_captions_mapping.values())
print("Total number of captions:", total_captions)

{"model_id":"lad50ef768e948d1969137dc4dbef63e","version_major":2,"version_minor":0}

```

Total number of captions: 40455

```

# Function for processing the captions
def clean(mapping):
    for key, captions in mapping.items():
        for i in range(len(captions)):
            # Take one caption at a time
            caption = captions[i]

```

```

        # Preprocessing steps
        # Convert to lowercase
        caption = caption.lower()
        # Remove non-alphabetical characters
        caption = ''.join(char for char in caption if
char.isalpha() or char.isspace())
        # Remove extra spaces
        caption = caption.replace('\s+', ' ')
        # Add unique start and end tokens to the caption
        caption = 'startseq ' + ' '.join([word for word in
caption.split() if len(word) > 1]) + ' endseq'
        captions[i] = caption

# before preprocess of text
image_to_captions_mapping['1026685415_0431cbf574']

['A black dog carries a green toy in his mouth as he walks through the
grass .',
'A black dog carrying something through the grass .',
'A black dog has a blue toy in its mouth .',
'A dog in grass with a blue item in his mouth .',
'A wet black dog is carrying a green toy through the grass .']

# preprocess the text
clean(image_to_captions_mapping)

# after preprocess of text
image_to_captions_mapping['1026685415_0431cbf574']

['startseq black dog carries green toy in his mouth as he walks
through the grass endseq',
'startseq black dog carrying something through the grass endseq',
'startseq black dog has blue toy in its mouth endseq',
'startseq dog in grass with blue item in his mouth endseq',
'startseq wet black dog is carrying green toy through the grass
endseq']

# Creating a List of All Captions
all_captions = [caption for captions in
image_to_captions_mapping.values() for caption in captions]

all_captions[:10]

['startseq child in pink dress is climbing up set of stairs in an
entry way endseq',
'startseq girl going into wooden building endseq',
'startseq little girl climbing into wooden playhouse endseq',
'startseq little girl climbing the stairs to her playhouse endseq',
'startseq little girl in pink dress going into wooden cabin endseq',
'startseq black dog and spotted dog are fighting endseq',
'startseq black dog and tricolored dog playing with each other on the

```

```

road endseq',
'startseq black dog and white dog with brown spots are staring at
each other in the street endseq',
'startseq two dogs of different breeds looking at each other on the
road endseq',
'startseq two dogs on pavement moving toward each other endseq']

# Tokenizing the Text
tokenizer = Tokenizer()
tokenizer.fit_on_texts(all_captions)

# Save the tokenizer
with open('tokenizer.pkl', 'wb') as tokenizer_file:
    pickle.dump(tokenizer, tokenizer_file)

# Load the tokenizer
with open('tokenizer.pkl', 'rb') as tokenizer_file:
    tokenizer = pickle.load(tokenizer_file)

# Calculate maximum caption length
max_caption_length = max(len(tokenizer.texts_to_sequences([caption])
[0]) for caption in all_captions)
vocab_size = len(tokenizer.word_index) + 1

# Print the results
print("Vocabulary Size:", vocab_size)
print("Maximum Caption Length:", max_caption_length)

Vocabulary Size: 8768
Maximum Caption Length: 34

# Creating a List of Image IDs
image_ids = list(image_to_captions_mapping.keys())
# Splitting into Training and Test Sets
split = int(len(image_ids) * 0.90)
train = image_ids[:split]
test = image_ids[split:]

# Data generator function
def data_generator(data_keys, image_to_captions_mapping, features,
tokenizer, max_caption_length, vocab_size, batch_size):
    # Lists to store batch data
    X1_batch, X2_batch, y_batch = [], [], []
    # Counter for the current batch size
    batch_count = 0

    while True:
        # Loop through each image in the current batch
        for image_id in data_keys:
            # Get the captions associated with the current image
            captions = image_to_captions_mapping[image_id]

```

```

# Loop through each caption for the current image
for caption in captions:
    # Convert the caption to a sequence of token IDs
    caption_seq = tokenizer.texts_to_sequences([caption])

[0]

    # Loop through the tokens in the caption sequence
    for i in range(1, len(caption_seq)):
        # Split the sequence into input and output pairs
        in_seq, out_seq = caption_seq[:i], caption_seq[i]

        # Pad the input sequence to the specified maximum
caption length
        in_seq = pad_sequences([in_seq],
maxlen=max_caption_length)[0]

        # Convert the output sequence to one-hot encoded
format
        out_seq = to_categorical([out_seq],
num_classes=vocab_size)[0]

        # Append data to batch lists
X1_batch.append(features[image_id][0]) # Image
features

        X2_batch.append(in_seq) # Input sequence
        y_batch.append(out_seq) # Output sequence

        # Increase the batch counter
        batch_count += 1

        # If the batch is complete, yield the batch and
reset lists and counter
        if batch_count == batch_size:
            X1_batch, X2_batch, y_batch =
np.array(X1_batch), np.array(X2_batch), np.array(y_batch)
            yield [X1_batch, X2_batch], y_batch
            X1_batch, X2_batch, y_batch = [], [], []
            batch_count = 0

# Encoder model
inputs1 = Input(shape=(4096,))
fe1 = Dropout(0.5)(inputs1)
fe2 = Dense(256, activation='relu')(fe1)
fe2_projected = RepeatVector(max_caption_length)(fe2)
fe2_projected = Bidirectional(LSTM(256, return_sequences=True))
(fe2_projected)

# Sequence feature layers
inputs2 = Input(shape=(max_caption_length,))

```

```

se1 = Embedding(vocab_size, 256, mask_zero=True)(inputs2)
se2 = Dropout(0.5)(se1)
se3 = Bidirectional(LSTM(256, return_sequences=True))(se2)

# Apply attention mechanism using Dot product
attention = Dot(axes=[2, 2])([fe2_projected, se3]) # Calculate
attention_scores

# Softmax attention scores
attention_scores = Activation('softmax')(attention)

# Apply attention scores to sequence embeddings
attention_context = Lambda(lambda x: tf.einsum('ijk,ijl->ikl', x[0],
x[1]))([attention_scores, se3])

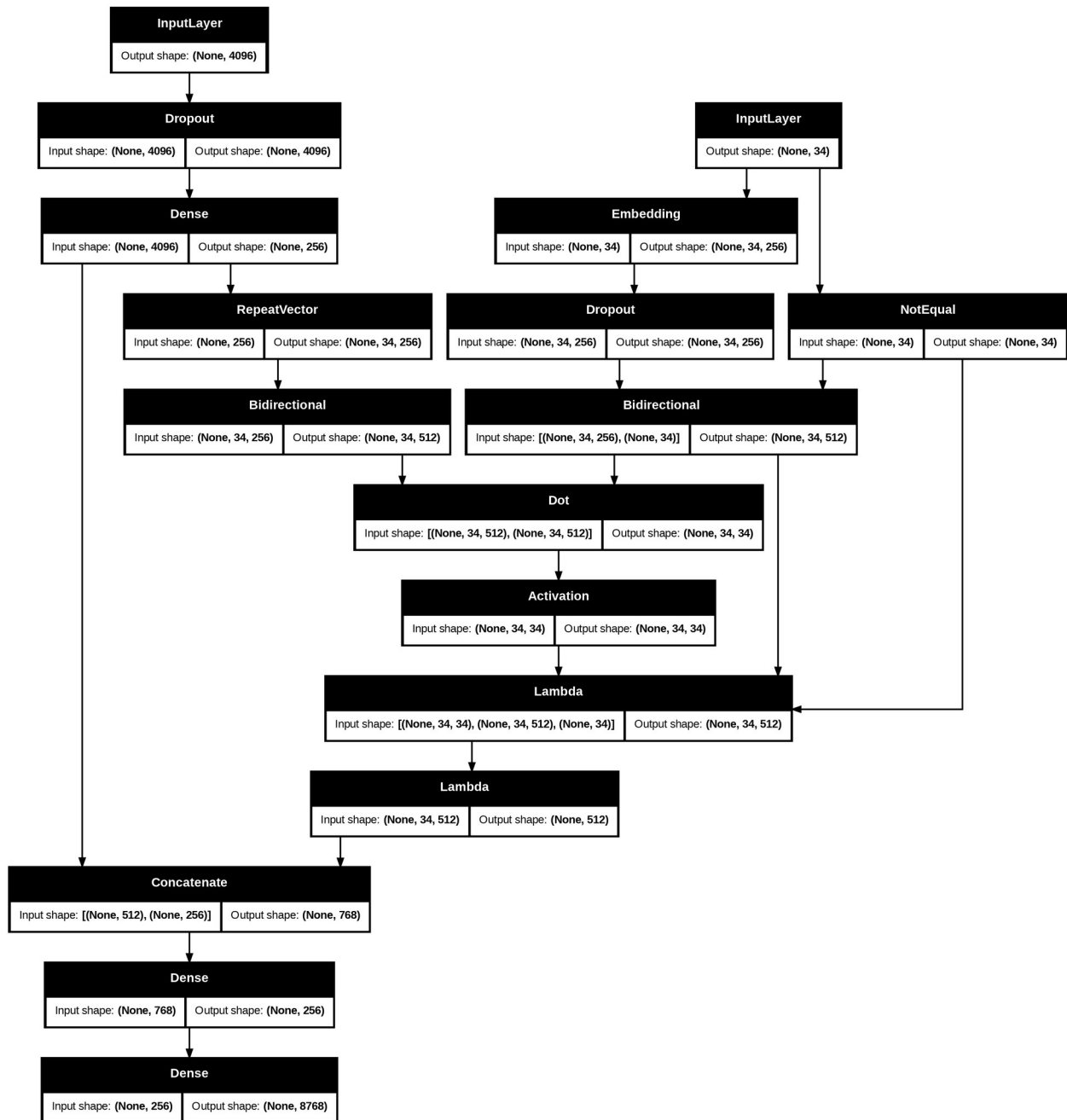
# Sum the attended sequence embeddings along the time axis
# Wrap tf.reduce_sum in a Lambda layer
context_vector = Lambda(lambda x: tf.reduce_sum(x, axis=1))
(context_vector)

# Decoder model
decoder_input = concatenate([context_vector, fe2], axis=-1)
decoder1 = Dense(256, activation='relu')(decoder_input)
outputs = Dense(vocab_size, activation='softmax')(decoder1)

# Create the model
model = Model(inputs=[inputs1, inputs2], outputs=outputs)
model.compile(loss='categorical_crossentropy', optimizer='adam')

# Visualize the model
plot_model(model, show_shapes=True)

```



```
# Assuming train_generator and test_generator are yielding the correct
format
```

```
for epoch in range(epochs):
    print(f"Epoch {epoch+1}/{epochs}")
```

```
# Set up data generators
```

```
train_generator = data_generator(train, image_to_captions_mapping,
loaded_features, tokenizer, max_caption_length, vocab_size,
batch_size)
```

```
test_generator = data_generator(test, image_to_captions_mapping,
```

```
loaded_features, tokenizer, max_caption_length, vocab_size,  
batch_size)
```

```
    # Train the model for one epoch  
    model.fit(train_generator, epochs=1,  
steps_per_epoch=steps_per_epoch,  
                validation_data=test_generator,  
validation_steps=validation_steps,  
                verbose=1)
```

Epoch 1/50

228/228 ————— 216s 882ms/step - loss: 6.8070 -

val_loss: 6.3171

Epoch 2/50

228/228 ————— 197s 863ms/step - loss: 5.1895 -

val_loss: 6.5837

Epoch 3/50

228/228 ————— 193s 847ms/step - loss: 4.8374 -

val_loss: 6.4335

Epoch 4/50

228/228 ————— 198s 867ms/step - loss: 4.4663 -

val_loss: 6.8408

Epoch 5/50

228/228 ————— 192s 840ms/step - loss: 4.3216 -

val_loss: 6.8418

Epoch 6/50

228/228 ————— 192s 844ms/step - loss: 3.9783 -

val_loss: 6.8926

Epoch 7/50

228/228 ————— 193s 847ms/step - loss: 3.5929 -

val_loss: 7.2545

Epoch 8/50

228/228 ————— 201s 878ms/step - loss: 3.3100 -

val_loss: 7.6845

Epoch 9/50

228/228 ————— 195s 855ms/step - loss: 3.0430 -

val_loss: 7.4958

Epoch 10/50

228/228 ————— 193s 849ms/step - loss: 2.8204 -

val_loss: 7.0688

Epoch 11/50

228/228 ————— 193s 846ms/step - loss: 2.6403 -

val_loss: 6.8822

Epoch 12/50

228/228 ————— 193s 847ms/step - loss: 2.5234 -

val_loss: 6.9444

Epoch 13/50

228/228 ————— 190s 834ms/step - loss: 2.4217 -

val_loss: 7.6048

Epoch 14/50

```
228/228 _____ 191s 837ms/step - loss: 2.2444 -  
val_loss: 7.2466  
Epoch 15/50  
228/228 _____ 193s 847ms/step - loss: 2.1131 -  
val_loss: 7.3402  
Epoch 16/50  
228/228 _____ 193s 847ms/step - loss: 1.9754 -  
val_loss: 7.0000  
Epoch 17/50  
228/228 _____ 199s 873ms/step - loss: 1.9684 -  
val_loss: 7.3665  
Epoch 18/50  
228/228 _____ 199s 871ms/step - loss: 1.8089 -  
val_loss: 7.6332  
Epoch 19/50  
228/228 _____ 192s 844ms/step - loss: 1.6989 -  
val_loss: 8.1608  
Epoch 20/50  
228/228 _____ 190s 836ms/step - loss: 1.5562 -  
val_loss: 8.0091  
Epoch 21/50  
228/228 _____ 194s 851ms/step - loss: 1.5220 -  
val_loss: 8.3351  
Epoch 22/50  
228/228 _____ 192s 844ms/step - loss: 1.4475 -  
val_loss: 8.2358  
Epoch 23/50  
228/228 _____ 194s 852ms/step - loss: 1.3738 -  
val_loss: 8.7327  
Epoch 24/50  
228/228 _____ 192s 838ms/step - loss: 1.2822 -  
val_loss: 9.0362  
Epoch 25/50  
228/228 _____ 189s 832ms/step - loss: 1.2475 -  
val_loss: 9.2237  
Epoch 26/50  
228/228 _____ 192s 842ms/step - loss: 1.2150 -  
val_loss: 9.6324  
Epoch 27/50  
228/228 _____ 190s 832ms/step - loss: 1.1618 -  
val_loss: 9.6000  
Epoch 28/50  
228/228 _____ 190s 833ms/step - loss: 1.0304 -  
val_loss: 9.5594  
Epoch 29/50  
228/228 _____ 191s 838ms/step - loss: 1.0025 -  
val_loss: 10.8196  
Epoch 30/50  
228/228 _____ 193s 845ms/step - loss: 0.9783 -
```

```
val_loss: 10.9478
Epoch 31/50
228/228 _____ 191s 839ms/step - loss: 0.9317 -
val_loss: 10.6557
Epoch 32/50
228/228 _____ 192s 839ms/step - loss: 0.8530 -
val_loss: 11.2696
Epoch 33/50
228/228 _____ 187s 821ms/step - loss: 0.8322 -
val_loss: 11.7002
Epoch 34/50
228/228 _____ 190s 836ms/step - loss: 0.7671 -
val_loss: 11.0039
Epoch 35/50
228/228 _____ 185s 813ms/step - loss: 0.7335 -
val_loss: 12.1717
Epoch 36/50
228/228 _____ 189s 830ms/step - loss: 0.7071 -
val_loss: 12.7301
228/228 _____ 187s 821ms/step - loss: 0.6523 -
val_loss: 12.8175
Epoch 38/50
228/228 _____ 188s 823ms/step - loss: 0.6455 -
val_loss: 12.8255
Epoch 39/50
228/228 _____ 189s 831ms/step - loss: 0.5687 -
val_loss: 12.5559
Epoch 40/50
228/228 _____ 190s 835ms/step - loss: 0.5403 -
val_loss: 13.0365
Epoch 41/50
228/228 _____ 188s 825ms/step - loss: 0.5489 -
val_loss: 13.4497
Epoch 42/50
228/228 _____ 187s 818ms/step - loss: 0.5183 -
val_loss: 14.0543
Epoch 43/50
228/228 _____ 191s 837ms/step - loss: 0.5421 -
val_loss: 13.5429
Epoch 44/50
228/228 _____ 191s 838ms/step - loss: 0.5638 -
val_loss: 13.2398
Epoch 45/50
228/228 _____ 191s 837ms/step - loss: 0.5304 -
val_loss: 14.4041
Epoch 46/50
228/228 _____ 192s 844ms/step - loss: 0.5175 -
val_loss: 14.8150
Epoch 47/50
```

```

228/228 _____ 191s 836ms/step - loss: 0.5155 -
val_loss: 14.2205
Epoch 48/50
228/228 _____ 187s 819ms/step - loss: 0.4912 -
val_loss: 14.7799
Epoch 49/50
228/228 _____ 189s 827ms/step - loss: 0.4758 -
val_loss: 13.4625
Epoch 50/50
228/228 _____ 190s 835ms/step - loss: 0.4781 -
val_loss: 13.9783

model.save(OUTPUT_DIR + '/mymodel.keras')

def get_word_from_index(index, tokenizer):
    return next((word for word, idx in tokenizer.word_index.items() if
idx == index), None)

def predict_caption(model, image_features, tokenizer,
max_caption_length):
    # Initialize the caption sequence
    caption = 'startseq'

    # Generate the caption
    for _ in range(max_caption_length):
        # Convert the current caption to a sequence of token indices
        sequence = tokenizer.texts_to_sequences([caption])[0]
        # Pad the sequence to match the maximum caption length
        sequence = pad_sequences([sequence],
maxlen=max_caption_length)
        # Predict the next word's probability distribution
        yhat = model.predict([image_features, sequence], verbose=0)
        # Get the index with the highest probability
        predicted_index = np.argmax(yhat)
        # Convert the index to a word
        predicted_word = get_word_from_index(predicted_index,
tokenizer)

        # Append the predicted word to the caption
        caption += " " + predicted_word

        # Stop if the word is None or if the end sequence tag is
encountered
        if predicted_word is None or predicted_word == 'endseq':
            break

    return caption

# Initialize lists to store actual and predicted captions
actual_captions_list = []
predicted_captions_list = []

```

```

# Loop through the test data
for key in tqdm(test):
    # Get actual captions for the current image
    actual_captions = image_to_captions_mapping[key]
    # Predict the caption for the image using the model
    predicted_caption = predict_caption(model, loaded_features[key],
tokenizer, max_caption_length)

    # Split actual captions into words
    actual_captions_words = [caption.split() for caption in
actual_captions]
    # Split predicted caption into words
    predicted_caption_words = predicted_caption.split()

    # Append to the lists
    actual_captions_list.append(actual_captions_words)
    predicted_captions_list.append(predicted_caption_words)

# Calculate BLEU score
print("BLEU-1: %f" % corpus_bleu(actual_captions_list,
predicted_captions_list, weights=(1.0, 0, 0, 0)))
print("BLEU-2: %f" % corpus_bleu(actual_captions_list,
predicted_captions_list, weights=(0.5, 0.5, 0, 0)))

{"model_id": "83a78127e41b46078ecaa95c9987b271", "version_major": 2, "vers
ion_minor": 0}

BLEU-1: 0.028607
BLEU-2: 0.000000

# Function for generating caption
def generate_caption(image_name):
    # load the image
    image_id = image_name.split('.')[0]
    img_path = os.path.join(INPUT_DIR, "/content/drive/MyDrive/Image
Caption Generator using Deep Learning on Flickr8K dataset/Images",
image_name)
    image = Image.open(img_path)
    captions = image_to_captions_mapping[image_id]
    print('-----Actual-----')
    for caption in captions:
        print(caption)
    # predict the caption
    y_pred = predict_caption(model, loaded_features[image_id],
tokenizer, max_caption_length)
    print('-----Predicted-----')
    print(y_pred)
    plt.imshow(image)

generate_caption("101669240_b2d3e7f17b.jpg")

```

```
-----Actual-----
startseq man in hat is displaying pictures next to skier in blue hat
endseq
startseq man skis past another man displaying paintings in the snow
endseq
startseq person wearing skis looking at framed pictures set up in the
snow endseq
startseq skier looks at framed pictures in the snow next to trees
endseq
startseq man on skis looking at artwork for sale in the snow endseq
-----Predicted-----
startseq man on skis looking at sale in the snow endseq
```

